

Práctica 3

Comunicaciones Inalámbricas

Alumnos: Francisco Nortes, Mario Martínez.

Profesor: Lenin Lemus.

Índice

1. Objetivo	2
2. Obtener temperatura DHT-11	3
3. Bluetooth	5
3.1. Servidor	5
3.2. Cliente	7
5. Conexión IoT a la UPV	9
5.1. WI-FI server	9
5.2. Código WI-FI server	10
6. Problemas encontrados	11
7. Bibliografía	12
8. Anexos	12
8.1. Código Bluetooth Server	12
8.2. Código Bluetooth Client	16
8.3. Código WI-FI server	20
8.4. Código WI-FI client	21

1. Objetivo

Para esta parte de la práctica, se planteó el diseño e implementación de dos aplicaciones destinadas a transmitir la temperatura ambiente del laboratorio a través de una red. La primera aplicación utiliza la tecnología Bluetooth BLE, mientras que la segunda emplea conectividad Wi-Fi, de manera que se puedan comparar las ventajas y limitaciones de cada método.

1.1. Material necesario:

Para llevar a cabo estas aplicaciones se requiere un ordenador, dos placas de desarrollo ESP32-S3-DevKitC-1, un sensor de temperatura DHT11 y acceso a la red IoT de la Universitat Politècnica de València.

2. Obtener temperatura DHT-11

Para este apartado se ha desarrollado un programa cuyo objetivo es obtener la temperatura medida por el sensor DHT11 y mostrarla a través del puerto serie.

El funcionamiento del código es sencillo. En primer lugar, se incluye la librería necesaria para trabajar con el sensor <DHT.h> y se define el pin al que está conectado (pin 2), así como el tipo de dispositivo utilizado. A continuación, se crea un objeto que permite gestionar la comunicación con el sensor.

Durante la ejecución del programa, en el bucle principal se realiza la lectura de la temperatura cada cierto tiempo. Antes de mostrar el valor, se comprueba que la lectura sea válida, ya que este tipo de sensores puede devolver errores ocasionales. Si la lectura es correcta, se imprime la temperatura en grados Celsius; en caso contrario, se muestra un mensaje de error.

Finalmente, se introduce una pequeña pausa entre lecturas para respetar el tiempo mínimo de muestreo recomendado por el sensor y asegurar un funcionamiento estable.

```
#include <DHT.h>

// Configuración del sensor
#define DHTPIN 2
#define DHTTYPE DHT11

DHT dht(DHTPIN, DHTTYPE);

void setup() {
  Serial.begin(115200);
  dht.begin();
  Serial.println("Sensor DHT11 inicializado.");
}
```

```

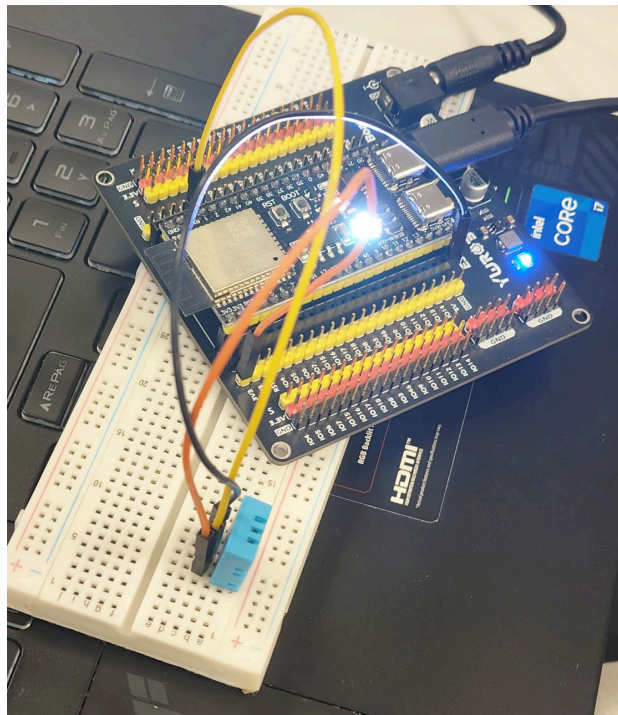
void loop() {
    // Leer temperatura
    float temperatura = dht.readTemperature();

    // Comprobar si la lectura es válida
    if (isnan(temperatura)) {
        Serial.println("Error leyendo el DHT11");
    } else {
        Serial.print("Temperatura: ");
        Serial.print(temperatura);
        Serial.println(" °C");
    }

    delay(2000); // Espera mínima recomendada para el DHT11
}

```

A continuación, se muestra como es el montaje que se llevó a cabo en el laboratorio con el sensor DHT11.



3. Bluetooth

3.1. Servidor

En esta parte de la práctica se comenzó a trabajar con tecnologías de comunicación inalámbrica, con el objetivo de transmitir la temperatura medida en el laboratorio. En lugar de enviar los datos directamente de una ESP a otra mediante cables, se va a utilizar BLE (Bluetooth Low Energy).

La idea es crear un servidor BLE en una de las ESP32-S3 que aloje el dato de temperatura. Otra ESP actuará como cliente y se conectará al servidor para leer este valor de forma inalámbrica. De esta manera, se logra transmitir la información del sensor sin necesidad de conexiones físicas.

En la fase de inicialización, se configura el sensor de temperatura y se establece el entorno BLE. Se crea un servidor, un servicio y dos características (characteristics) dentro de dicho servicio. Estas características permiten transmitir la información de temperatura en dos formatos distintos: uno como valor numérico y otro como cadena de texto descriptiva.

```
// UUIDs
#define SERVICE_UUID           "8628b276-4bba-420a-ab99-dd69945408bf"
#define CHARACTERISTIC_UUID_1 "6cb1114c-4410-4757-85af-b064b3cc689a"
#define CHARACTERISTIC_UUID_2 "a1642e22-4f4e-41de-b173-b5bdb099f12c"
```

En SERVICE_UUID identifica el servicio BLE, en nuestro caso representa el “servicio de temperatura”

- CHARACTERISTIC_UUID_1: Esta característica envía la temperatura como número
- CHARACTERISTIC_UUID_2: Envía la temperatura como texto

Además, se habilitan notificaciones para que los datos se envíen automáticamente al dispositivo conectado sin necesidad de solicitud explícita.

```
pBLE2902_1 = new BLE2902();
pBLE2902_1->setNotifications(true);
pCharacteristic_1->addDescriptor(pBLE2902_1);

pBLE2902_2 = new BLE2902();
pBLE2902_2->setNotifications(true);
pCharacteristic_2->addDescriptor(pBLE2902_2);
```

Y más tarde:

```
// ===== Enviar como valor numérico =====
pCharacteristic_1->setValue(temperatura);
pCharacteristic_1->notify();

// ===== Enviar como texto descriptivo =====
String mensaje = "Temperatura actual: " + String(temperatura) + " C";
pCharacteristic_2->setValue(mensaje.c_str());
pCharacteristic_2->notify();
```

Durante la ejecución del programa, la ESP32 permanece a la espera de una conexión BLE. Una vez establecida, el sistema realiza lecturas periódicas de la temperatura a través del sensor DHT11. Si la lectura es válida, el valor obtenido se envía mediante las características BLE configuradas, permitiendo su visualización remota. Este proceso se repite de forma continua con un intervalo de tiempo adecuado para las limitaciones del sensor.

```
float temperatura = dht.readTemperature();
if (isnan(temperatura)) {
    Serial.println("Error leyendo el DHT11");
    delay(2000);
    return;
}
```

Asimismo, el código incluye un mecanismo de gestión de conexiones, de modo que, si el dispositivo cliente se desconecta, la ESP32 vuelve automáticamente a modo de difusión (advertising), quedando disponible para nuevas conexiones.

```
if (!deviceConnected && oldDeviceConnected) {
    delay(500);
    pServer->startAdvertising();
    Serial.println("Reiniciando advertising");
    oldDeviceConnected = deviceConnected;
}
if (deviceConnected && !oldDeviceConnected) {
    oldDeviceConnected = deviceConnected;
}
```

En conjunto, el sistema permite monitorizar la temperatura de forma inalámbrica y en tiempo real, haciendo uso de las capacidades BLE de la ESP32.

3.2. Cliente

Para poder consultar el dato de la temperatura en la otra ESP-32-S3 se usó un código que escaneaba los dispositivos BLE cercanos hasta encontrar el servidor.

```
// UUIDs (igual que en el servidor)
#define SERVICE_UUID          "8628b276-4bba-420a-ab99-dd69945408bf"
#define CHARACTERISTIC_UUID   "a1642e22-4f4e-41de-b173-b5bdb099f12c"
```

1. Si dentro del servidor encuentra el servicio correcto, es decir, que coincida con el de la temperatura, guarda su dirección y activa la conexión.

```
if (advertisedDevice.haveServiceUUID() &&
    advertisedDevice.isAdvertisingService(BLEUUID(SERVICE_UUID))) {

    doConnect = true;
    advertisedDevice.getScan()->stop();
}
```

2. Se conecta a la ESP32 que envía la temperatura:

```
pClient->connect(*pServerAddress);
```

3. Accede al dato que contiene la temperatura, en este caso nos basta con el valor en formato de texto.

```
pRemoteCharacteristic =
pRemoteService->getCharacteristic(CHARACTERISTIC_UUID);
```


4. Y por último se suscribe para recibir los datos automáticamente sin necesidad de pedirlos.

```
registerForNotify(notifyCallback);
```

El código completo está en el anexo de la práctica.

5. Conexión IoT a la UPV

5.1. WI-FI server

En este apartado, se trabaja con la conectividad Wi-Fi para transmitir la temperatura del laboratorio a cualquier dispositivo interesado. Para ello, la ESP32-S3 se conecta a la red inalámbrica de la Universitat Politècnica de València mediante un SSID y una contraseña proporcionados por el profesor durante la clase.

```
const char* ssid = "UPV-PSK";
const char* password = "C0n3ct4nd0s3_m0l4!";
WebServer server(80);
```

Define la red wifi a la que se conectará la ESP32, red especial para este tipo de prácticas y crea un servidor web en el puerto 80 (HTTP estándar).

La función con la que enviamos la temperatura es la siguiente:

```
void handleTemperatura() {
    float temperatura = dht.readTemperature();
    if (isnan(temperatura)) {
        server.send(500, "text/plain", "Error leyendo sensor");
        return;
    }
    server.send(200, "text/plain", String(temperatura));
}
```

Se llama cada vez que alguien visita "http://IP_DEL_ESP32/temperatura"

Esta IP de la ESP se le es asignada automáticamente al conectarse a la red WI-FI "UPV-PSK". Luego la imprimimos por pantalla para saber, en el código del cliente, donde conseguir el dato de la temperatura.

```
Serial.println("\nConectado!");
Serial.print("IP del servidor: ");
Serial.println(WiFi.localIP());
```

Una vez hemos logrado la conexión, mediante “server.on”, indicamos que hacer al servidor cuando alguien se conecte a /temperatura, que en este caso es ejecutar la función handleTemperatura, que lee del DHT11 y devuelve la temperatura como texto.

5.2. Código WI-Fi server

En este código la ESP32 se conecta a la misma red que el servidor pero esta vez para acceder a la url donde se encuentra el dato. Para acceder a esta dirección simplemente se combina la IP del servidor con la ruta /temperatura

```
String url = String("http://") + serverIP + "/temperatura";  
http.begin(url);
```

Una vez accede a la dirección de la temperatura, realiza la petición HTTP GET pidiéndole al servidor el valor del sensor.

```
int httpResponseCode = http.GET();
```

Si la lectura se ha realizado correctamente, se muestra por el monitor

```
if (httpResponseCode > 0) {  
    String temperatura = http.getString();  
    Serial.print("Temperatura recibida: ");  
    Serial.println(temperatura + " °C");  
} else {  
    Serial.print("Error en la conexión: ");  
    Serial.println(httpResponseCode);  
}
```

Todo ello se repite periódicamente cada 2 segundos.

6.Problemas encontrados

Durante la práctica, conectarse a la red IoT de la Universitat Politècnica de València resultó en ocasiones bastante complicado. Frecuentemente aparecían timeouts que obligaban a reiniciar el proceso de conexión. De hecho, algunos compañeros no lograban conectar sus ESP al Wi-Fi en absoluto.

Tras analizar la situación, llegamos a la conclusión de que la red tiene un sistema de control de acceso que mantiene una lista de dispositivos admitidos. Algunos equipos quedaban en espera dentro de esta lista, lo que impedía que se conectaran correctamente.

Como alternativa, se optó por utilizar el hotspot de un teléfono móvil. Esto permitió controlar directamente qué dispositivos podían conectarse, agilizando el proceso y asegurando que la práctica pudiera realizarse sin depender de las limitaciones de la red de la universidad. Esta solución demostró ser más fiable y rápida para el desarrollo de los experimentos.

7. Bibliografía

Material de apoyo a la docencia de PoliformaT.

OpenAI. (2026). ChatGPT (modelo de lenguaje).

8. Anexos

8.1. Código Bluetooth Server

```
#include <BLEDevice.h>
#include <BLEServer.h>
#include <BLEUtils.h>
#include <BLE2902.h>
#include <DHT.h>

// ===== CONFIGURACIÓN DHT =====

#define DHTPIN 2
#define DHTTYPE DHT11

DHT dht(DHTPIN, DHTTYPE);

// ===== BLE =====
BLEServer* pServer = NULL;
BLECharacteristic* pCharacteristic_1 = NULL;
BLECharacteristic* pCharacteristic_2 = NULL;
BLEDescriptor *pDescr_1;
BLE2902 *pBLE2902_1;
BLE2902 *pBLE2902_2;
```

```

bool deviceConnected = false;
bool oldDeviceConnected = false;

// UUIDs
#define SERVICE_UUID          "8628b276-4bba-420a-ab99-dd69945408bf"
#define CHARACTERISTIC_UUID_1 "6cb1114c-4410-4757-85af-b064b3cc689a"
#define CHARACTERISTIC_UUID_2 "a1642e22-4f4e-41de-b173-b5bdb099f12c"

class MyServerCallbacks: public BLEServerCallbacks {
    void onConnect(BLEServer* pServer) {
        deviceConnected = true;
    };

    void onDisconnect(BLEServer* pServer) {
        deviceConnected = false;
    }
};

void setup() {

    Serial.begin(115200);

    // Inicializar DHT
    dht.begin();
    Serial.println("Sensor DHT11 inicializado.");

    // Inicializar BLE
    BLEDevice::init("ESP32_Temperatura");

    pServer = BLEDevice::createServer();
    pServer->setCallbacks(new MyServerCallbacks());

```

```

BLEService *pService = pServer->createService(SERVICE_UUID);

pCharacteristic_1 = pService->createCharacteristic(
    CHARACTERISTIC_UUID_1,
    BLECharacteristic::PROPERTY_NOTIFY
);

pCharacteristic_2 = pService->createCharacteristic(
    CHARACTERISTIC_UUID_2,
    BLECharacteristic::PROPERTY_READ |
    BLECharacteristic::PROPERTY_WRITE |
    BLECharacteristic::PROPERTY_NOTIFY
);

pDescr_1 = new BLEDescriptor((uint16_t)0x2901);
pDescr_1->setValue("Temperatura en grados Celsius");
pCharacteristic_1->addDescriptor(pDescr_1);

pBLE2902_1 = new BLE2902();
pBLE2902_1->setNotifications(true);
pCharacteristic_1->addDescriptor(pBLE2902_1);

pBLE2902_2 = new BLE2902();
pBLE2902_2->setNotifications(true);
pCharacteristic_2->addDescriptor(pBLE2902_2);

pService->start();

BLEAdvertising *pAdvertising = BLEDevice::getAdvertising();
pAdvertising->addServiceUUID(SERVICE_UUID);
BLEDevice::startAdvertising();

```

```

    Serial.println("Esperando conexión BLE...");
}

void loop() {

    if (deviceConnected) {

        // Leer temperatura
        float temperatura = dht.readTemperature();

        if (isnan(temperatura)) {
            Serial.println("Error leyendo el DHT11");
            delay(2000);
            return;
        }

        Serial.print("Temperatura: ");
        Serial.println(temperatura);

        // ===== Enviar como valor numérico =====
        pCharacteristic_1->setValue(temperatura);
        pCharacteristic_1->notify();

        // ===== Enviar como texto descriptivo =====
        String mensaje = "Temperatura actual: " + String(temperatura) + " C";
        pCharacteristic_2->setValue(mensaje.c_str());
        pCharacteristic_2->notify();

        delay(2000); // DHT11 necesita mínimo 1s
    }

    // Manejo reconexión

```



```

if (!deviceConnected && oldDeviceConnected) {
    delay(500);
    pServer->startAdvertising();
    Serial.println("Reiniciando advertising");
    oldDeviceConnected = deviceConnected;
}

if (deviceConnected && !oldDeviceConnected) {
    oldDeviceConnected = deviceConnected;
}
}

```

8.2. Código Bluetooth Client

```

#include <BLEDevice.h>

#include <BLEUtils.h>

#include <BLEClient.h>

#include <BLERemoteCharacteristic.h>

// UUIDs (igual que en el servidor)

#define SERVICE_UUID          "8628b276-4bba-420a-ab99-dd69945408bf"

#define CHARACTERISTIC_UUID   "a1642e22-4f4e-41de-b173-b5bdb099f12c"


static BLEAddress *pServerAddress;

static BLERemoteCharacteristic* pRemoteCharacteristic;

static boolean doConnect = false;

```

```

// Callback cuando se reciben datos

static void notifyCallback(

    BLERemoteCharacteristic* pBLERemoteCharacteristic,

    uint8_t* pData,

    size_t length,

    bool isNotify) {

    String mensaje = "";

    for (int i = 0; i < length; i++) {

        mensaje += (char)pData[i];

    }

    Serial.print("Mensaje recibido: ");

    Serial.println(mensaje);

}

// Detectar el servidor BLE

class MyAdvertisedDeviceCallbacks: public BLEAdvertisedDeviceCallbacks {

    void onResult(BLEAdvertisedDevice advertisedDevice) {

        if (advertisedDevice.haveServiceUUID() &&

            advertisedDevice.isAdvertisingService(BLEUUID(SERVICE_UUID))) {

```

```

        Serial.println("Servidor encontrado");

        pServerAddress = new BLEAddress(advertisedDevice.getAddress());

        doConnect = true;

        advertisedDevice.getScan()->stop();

    }

}

};

void setup() {

    Serial.begin(115200);

    BLEDevice::init("");

    // Iniciar escaneo

    BLEScan* pBLEScan = BLEDevice::getScan();

    pBLEScan->setAdvertisedDeviceCallbacks(new
MyAdvertisedDeviceCallbacks());

    pBLEScan->setActiveScan(true);

    pBLEScan->start(30);

}

```

```

void loop() {

    if (doConnect) {

        BLEClient* pClient = BLEDevice::createClient();

        pClient->connect(*pServerAddress);

        Serial.println("Conectado al servidor");

        BLERemoteService* pRemoteService = pClient->getService(SERVICE_UUID);

        pRemoteCharacteristic =
        pRemoteService->getCharacteristic(CHARACTERISTIC_UUID);

        // Activar notificaciones

        if (pRemoteCharacteristic->canNotify()) {

            pRemoteCharacteristic->registerForNotify(notifyCallback);

        }

        doConnect = false;

    }

    delay(1000);

```

```
}
```

8.3. Código WI-FI server

```
#include <WiFi.h>
#include <WebServer.h>
#include <DHT.h>

#define DHTPIN 2
#define DHTTYPE DHT11

DHT dht(DHTPIN, DHTTYPE);

const char* ssid = "UPV-PSK";
const char* password = "C0n3ct4nd0s3_m0l4!";

WebServer server(80);

void handleTemperatura() {
    float temperatura = dht.readTemperature();

    if (isnan(temperatura)) {
        server.send(500, "text/plain", "Error leyendo sensor");
        return;
    }

    server.send(200, "text/plain", String(temperatura));
}

void setup() {
    Serial.begin(115200);
```

```

dht.begin();

WiFi.begin(ssid, password);
Serial.print("Conectando a WiFi");

while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
}

Serial.println("\nConectado!");
Serial.print("IP del servidor: ");
Serial.println(WiFi.localIP());

server.on("/temperatura", handleTemperatura);
server.begin();
}

void loop() {
    server.handleClient();
}

```

8.4. Código WI-FI client

```

#include <WiFi.h>
#include <HTTPClient.h>

const char* ssid = "UPV-PSK";
const char* password = "C0n3ct4nd0s3_m0l4!";

// IP del servidor que envía la temperatura
const char* serverIP = "192.168.1.42";

```

```

void setup() {
    Serial.begin(115200);

    // Conectar a WiFi
    WiFi.begin(ssid, password);
    Serial.print("Conectando a WiFi");
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }
    Serial.println("\nConectado a la red WiFi");
}

void loop() {
    if (WiFi.status() == WL_CONNECTED) {
        HTTPClient http;

        String url = String("http://") + serverIP + "/temperatura";
        http.begin(url);

        int httpResponseCode = http.GET();

        if (httpResponseCode > 0) {
            String temperatura = http.getString();
            Serial.print("Temperatura recibida: ");
            Serial.println(temperatura + " °C");
        } else {
            Serial.print("Error en la conexión: ");
            Serial.println(httpResponseCode);
        }
    }
}

```

```
    http.end();  
  } else {  
    Serial.println("WiFi desconectado");  
  }  
  
  delay(2000); // Leer cada 2 segundos  
}
```